Module 4 Narrative

William Mitchem

Bachelor of Science in Computer Science, SNHU

CS 499: Computer Science Capstone

Dr. Satish Penmatsa

June 5th, 2026

Module 4 Narrative

The artifact I chose was the full-stack SPA that I created as part of CS 465: Full Stack Development in the term before this one. The project consisted of a legacy Angular frontend, Node/Express backend, and a MongoDB database. Its purpose was to market trips offered by the Travlr travel agency.

Artifact Justification

I chose this artifact because it's expansive enough to offer opportunities for meeting all three enhancements, while also affording me the opportunity to shift the purpose of the software into what is, for me, a passion project. It's also the kind of work I want to do after graduation. That's why my concentration is in Software Engineering as well. I'll explain my justification further as I break down the outcomes in the next section.

Course Outcomes

I'm confident that I've met the course outcomes, as well as meeting my own planned outcomes from the first module. To that end, I'll break down how each outcome is met by my work.

Algorithmic Principles and Problem-Solving Outcome

The Dynamic Probability Calculator

I didn't want to just use simple, static numbers for the shiny hunting feature. Instead, I researched the actual math behind methods like "Poké Radar chaining," where the odds change with every successful encounter (chain). I wrote an algorithm that calculates the cumulative probability of finding a shiny Pokémon over a series of varying events. It proves I can take a complex, real-world mathematical concept and translate it into working code.

The Custom Filtering System (CollectionQuery)

Instead of just doing a basic alphabetical sort, I built a custom filtering engine. Because the application handles a large amount of Pokémon data, I needed a way to sift through it efficiently. I wrote a system that acts like a series of funnels, allowing the user to chain together different filters (like picking a specific generation *and* a specific type) to instantly sort the data without slowing down the application.

Software Design and Engineering

When building the filtering system, I originally designed it just to sort Pokémon. However, I realized that hardcoding it to only accept Pokémon data was a poor design choice. Instead, I used TypeScript 'generics' to make the CollectionQuery a universal tool. This means the exact same piece of code can now be used to filter a list of Users, a list of Trips, or a list of Pokémon without needing to be rewritten. This demonstrates my understanding of the DRY (Don't Repeat Yourself) principle and shows that I can write flexible, reusable code rather than just quick, single-use scripts.

Managing Design Trade-offs Outcome

I had to make a few specific choices about how the app handled data, which meant weighing trade-offs.

The Dynamic Probability Calculator

The trade-off here was between simplicity and authenticity. It would have been much easier to just use flat odds for every hunting method, but it wouldn't be accurate to how the games actually work. I chose to spend the extra time building the complex math logic to make the app genuinely useful for users.

The Custom Filtering System (CollectionQuery)

For the data filtering, I had to choose between filtering the data on the server or filtering it directly on the user's device. I chose to handle the filtering on the device (client-side) after the initial load, which makes the search bar feel lightning-fast, rather than making the user wait for a loading screen every time they type a letter.

Innovative Techniques and Practices Outcome

This outcome is about using the right tools to build a smart, well-constructed application. I demonstrated this by how I managed the data coming from the external PokéAPI.

PokéAPI Mapping

The API sends back an overwhelming amount of raw data for each Pokémon, most of which my app doesn't need. Instead of just dumping all of that data into the app and wasting memory, I used a "mapping" technique. I wrote a script that intercepts the raw data and trims the fat, picking out only the specific text and image URLs my app requires before saving it. This keeps the application running smoothly and shows that I understand how to manage data responsibly.

Professional Communication Outcome

I made sure that the logic powering these algorithms wasn't just a messy block of code. I used TypeScript to clearly define what my data was supposed to look like, and I added detailed, plain-English comments above my math functions. This ensures that if another student or professor looks at my project, they can easily follow my thought process and understand exactly how the math works without having to guess.

I also applied the concept of professional communication directly to the codebase itself. Because I made the CollectionQuery a generic, reusable tool, I knew other developers (or

evaluators) would need to understand how to apply it to their own data sets. To communicate this clearly, I wrote detailed TSDoc comments above the function. Instead of just leaving a messy block of code, I provided plain-English descriptions of what the function does, what parameters it requires, and gave an example of how to use it. This ensures my code is 'self-documenting' and communicates my architectural intent clearly to any peer who might work on the project after me.

Reflection & Challenges

I want to lead this reflection by stating the biggest challenge I've had, unrelated to coding or design or architecture, has been time. I'm submitting this enhancement late, since I'm using the same artifact for each enhancement, and because my program is built around an external API, I had to implement database and data structure/algorithmic enhancements at the same time to have a functioning product for review.

While implementing the PokémonDatabase component, which is paginated, I encountered issues with changing pages. The element would be constantly mounted and unmounted, spamming the API with get requests. I did some research and learned that the problem was my use of the Suspense component with React's useMemo hook. From my understanding, this led to a decoupling between the component's synchronous render passes and the asynchronous request lifecycle. Effectively, it created an infinite loop of rendering, destroying the existing data, fetching it from the server, and repeating. To solve the issue while maintaining expected functionality, I abandoned use of Suspense, which was eliminated in necessity by my client-side sorting.

On that subject, originally, I utilized server-side pagination to handle the pages and determine which Pokémon would be on which page. But that ultimately led to excessive server

requests, because I want the data to be highly searchable. At times the UI would be sluggish as well. To solve that, I swapped to client-side querying using TanStack Query in conjunction with my custom CollectionQuery utility to keep the data alive and updated (TanStack), while having efficient, quick-loading searching and filtering (CollectionQuery). I ran the numbers to see how much space it'd take to keep the entire Pokémon database cached in-memory, and it amounted to roughly 1.5Mb, which is minimal on a modern device. So overall, that provided a sweet spot between agreeability with the React lifecycle and fast page loading.

While testing the PokéDex page and using React's useState, I noticed that if I clicked a Pokémon image to go to the detailed Pokémon page and then clicked the browser's back button to return to the PokéDex overview page, the state of the page was entirely lost in terms of what filters were active before leaving the page. The root cause, I found out, was useState: when React unmounts the PokéDex page, the useState memory is cleared. I looked at the React documentation for some alternative hooks, and decided to swap to React Router's useSearchParams to integrate the filters into the browser's native history stack. I should've used that hook to begin with, but filters were a progressively added idea to the page, so I fell into a bad habit of just extending the pattern of useState I was already using to my newly added filters.

A major problem area was that, because I translated the original code to React, the project included Bootstrap. For this second enhancement, I added Tailwind to allow for more modern functionality, and that led to running into endless styling conflicts between the two. As an example, I kept running into issues with my styles not applying seemingly at random, most notably when swapping from light to dark theme, and I noticed it happened only on elements styled with Bootstrap's bg-light class. After examining the Bootstrap documentation, I realized the bg-light class (and all the bg classes) are internally marked important. So, when I added

dark/light theme compatibility through Tailwind, any element using that class refused to update its styling. To simplify the program (and drop a dependency), I just swapped entirely to Tailwind and reworked all components relying on Bootstrap classes.

Overall, with enhancement 2, I feel I've learned much more about the mechanisms of React and the architecture it provides, especially around hooks and state management. I got to know React in a less in-depth way with the first enhancement since I was swapping simple Angular components over to React. But now that I've worked on what I'd consider to be more complex features, I can easily say that I feel I've learned a ton over this phase of the project.

As a final note, I'm currently investigating ways to also implement the app as a static site that utilizes Firefox and Chrome's IndexedDB to eliminate the need for accounts and external databases for testing the site and making it offline compatible, while avoiding hosting fees. I think that's a value add because it'd help eliminate any friction for prospective employers who want to go beyond my portfolio and test it live, with no installations necessary. I would greatly appreciate feedback on that idea, though it's an initiative beyond the capstone in specific. That idea is not a replacement, either; it's an added option on top of the account-based version I'm implementing currently for the capstone.